

SOCKET PROGRAMMING

Networking Protocols

Lecturer: Chung Thùy Linh · ctlinh@fit.hcmus.edu.vn

Table of Contents

Agenda

01

OSI & TCP/IP Model · TCP vs UDP

02

TCP Header · UDP Header · FTP Header

03

Flow Control · Congestion Control · Ordering & Loss

04

Socket Introduction · Building a Client–Server App

05

Python socket – Core Functions

06

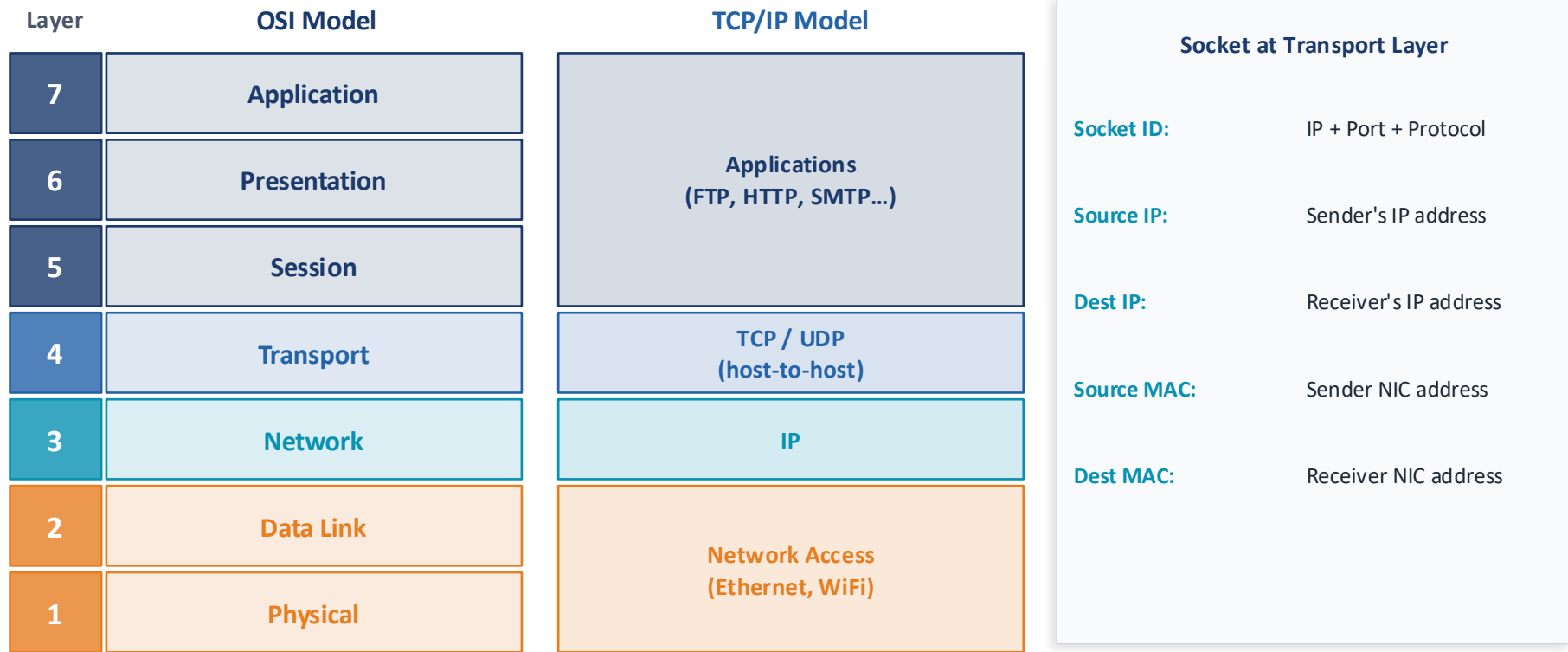
FTP Protocol – How It Works

07

Demo: TCP Chat · UDP Chat (Python)

OSI & TCP/IP Model

Layered Network Architecture



TCP vs UDP – Comparison

Transport Layer Protocols

TCP – Transmission Control Protocol

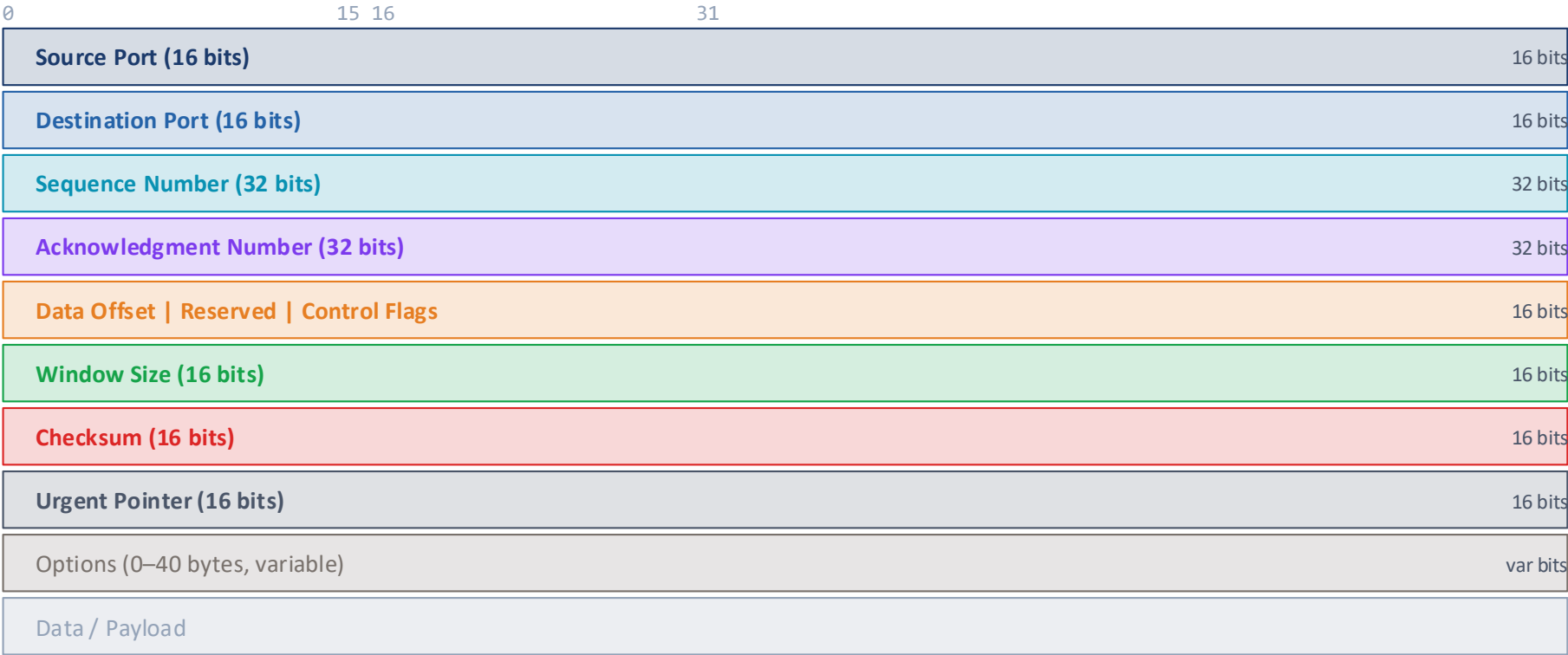
Connection	Connection-oriented (3-way handshake)
Reliability	Guaranteed delivery (Reliable)
Ordering	Ordered packet delivery
Speed	Slower (due to overhead)
Control	Flow + Congestion Control
Cast type	Unicast only
Use cases	HTTP/S, Email, FTP, SSH
✓ Reliable	
✓ Ordered	
✓ Congestion Control	

UDP – User Datagram Protocol

Connectionless (No setup required)
No guarantee (Best-effort)
No ordering guarantee
Faster (minimal overhead)
None
Unicast, Multicast, Broadcast
VoIP, DNS, Video Streaming, Gaming
⚡ High Speed
⚡ Real-time Friendly
⚡ Multicast/Broadcast

TCP Header – Packet Structure

Minimum 20 bytes · Transport Layer



Control Flags: URG · ACK · PSH · RST · SYN · FIN (1 bit each)

UDP Header – Packet Structure

Fixed 8 bytes · Simple & Fast

Source Port (16 bits)

Source port number (optional, = 0 if not used)

Destination Port (16 bits)

Destination port number

Length (16 bits)

Total length of UDP header + data (in bytes)

Checksum (16 bits)

Error-checking (optional in IPv4, mandatory in IPv6)

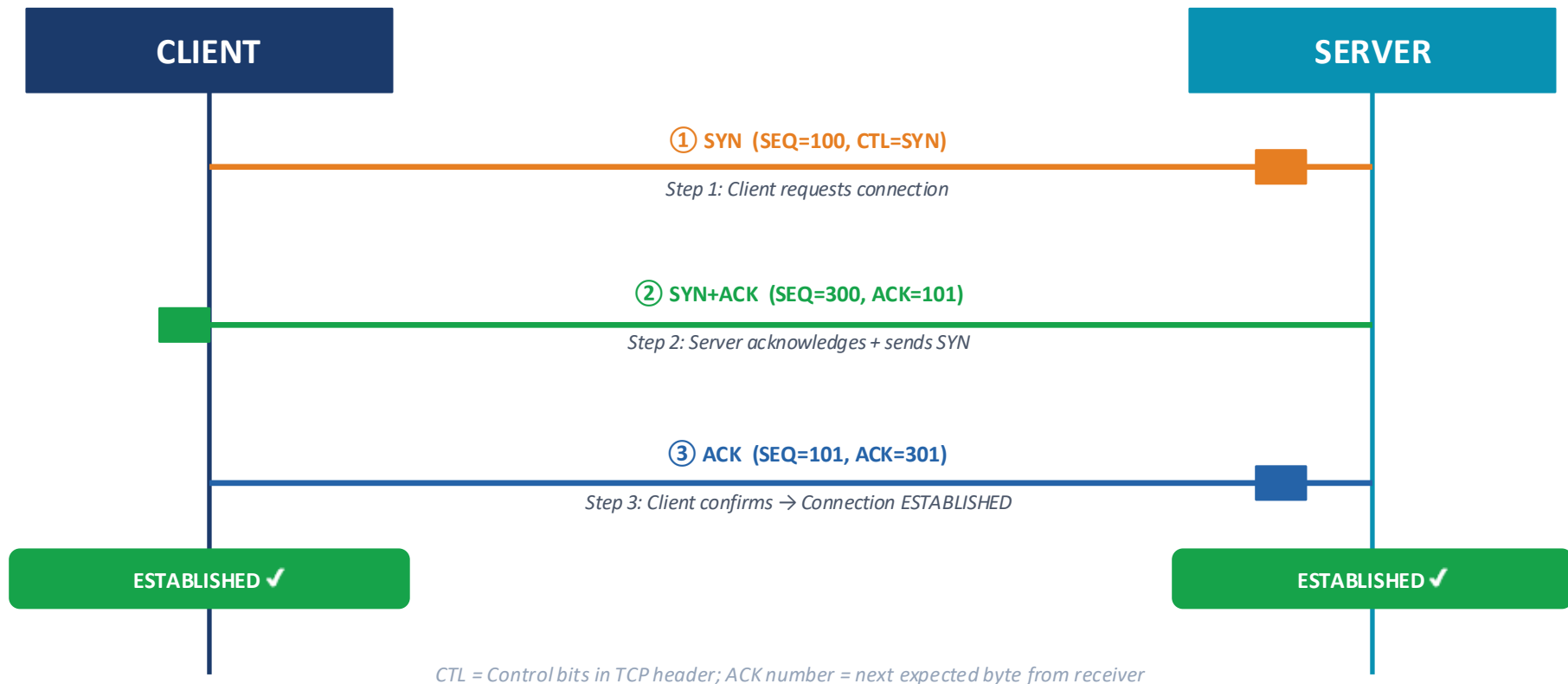
Data / Payload

Application data

💡 UDP header is only 8 bytes (vs TCP's minimum 20 bytes). No sequence numbers, ACK or flow control → faster but unreliable. Best for: VoIP, DNS, video streaming, gaming.

TCP – Connection Establishment (3-Way Handshake)

SYN · SYN-ACK · ACK



TCP – Flow Control

Preventing receiver buffer overflow

How It Works

1 Receive Window (rwnd)

Receiver advertises remaining buffer space in every ACK

2 Sender adjusts

Sender must not have more than rwnd unacknowledged bytes in flight

3 Window = 0

Sender pauses; waits for receiver to announce new window

4 Sliding Window

Window 'slides' forward with each ACK received

Sliding Window – Illustration



Sent & ACK'd



Sent, awaiting ACK (in window)



Not yet sent

Send Condition:
 $\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$

⚠ Window size is 16 bits → max 65,535 bytes; use Window Scale Option (RFC 1323) to extend

TCP – Congestion Control

Preventing network congestion

Slow Start

cwnd starts at 1 MSS
Doubles every RTT
until reaching ssthresh

Congestion Avoidance

cwnd increases linearly
(+1 MSS/RTT)
when $cwnd \geq ssthresh$

Fast Retransmit

3 duplicate ACKs →
retransmit lost packet
immediately (no timeout)

Fast Recovery

$ssthresh = cwnd/2$
 $cwnd = ssthresh$
Resume Congestion Avoidance

TCP – Packet Ordering & Loss Recovery

Sequence Numbers · Retransmission · ARQ

Packet Ordering

1. Each TCP segment carries a Sequence Number (SEQ) identifying the first byte of its payload.
2. Receiver uses SEQ to reorder out-of-order segments into the correct sequence.
3. Early-arriving segments are buffered until the missing segment arrives.
4. Receiver sends ACK = SEQ of the next expected byte (Cumulative ACK).
5. Once buffer is complete and ordered, data is passed up to the application layer.

Loss Detection & Recovery

Timeout (RTO)

Sender starts a timer; if it expires before ACK arrives → retransmit all segments from the lost one.

3 Duplicate ACKs (Fast Retransmit)

On 3 duplicate ACKs → immediately retransmit without waiting for timeout.

Selective ACK (SACK)

Extension allowing receiver to report exactly which segments arrived, avoiding redundant retransmissions.

Go-Back-N vs Selective Repeat

TCP uses a variant of Selective Repeat to optimize network performance.

Socket Introduction

Socket Programming · Winsock API · POSIX



What is a Socket?

An end-point of a communication link between two networked applications. It provides an interface for network programming at the Transport layer.



Socket Identity

A socket is uniquely identified by: IP Address + Port Number + Protocol (TCP/UDP)



Winsock API

Windows Socket API – supports building TCP/IP network applications on Windows. MFC provides the CSocket class as a wrapper.

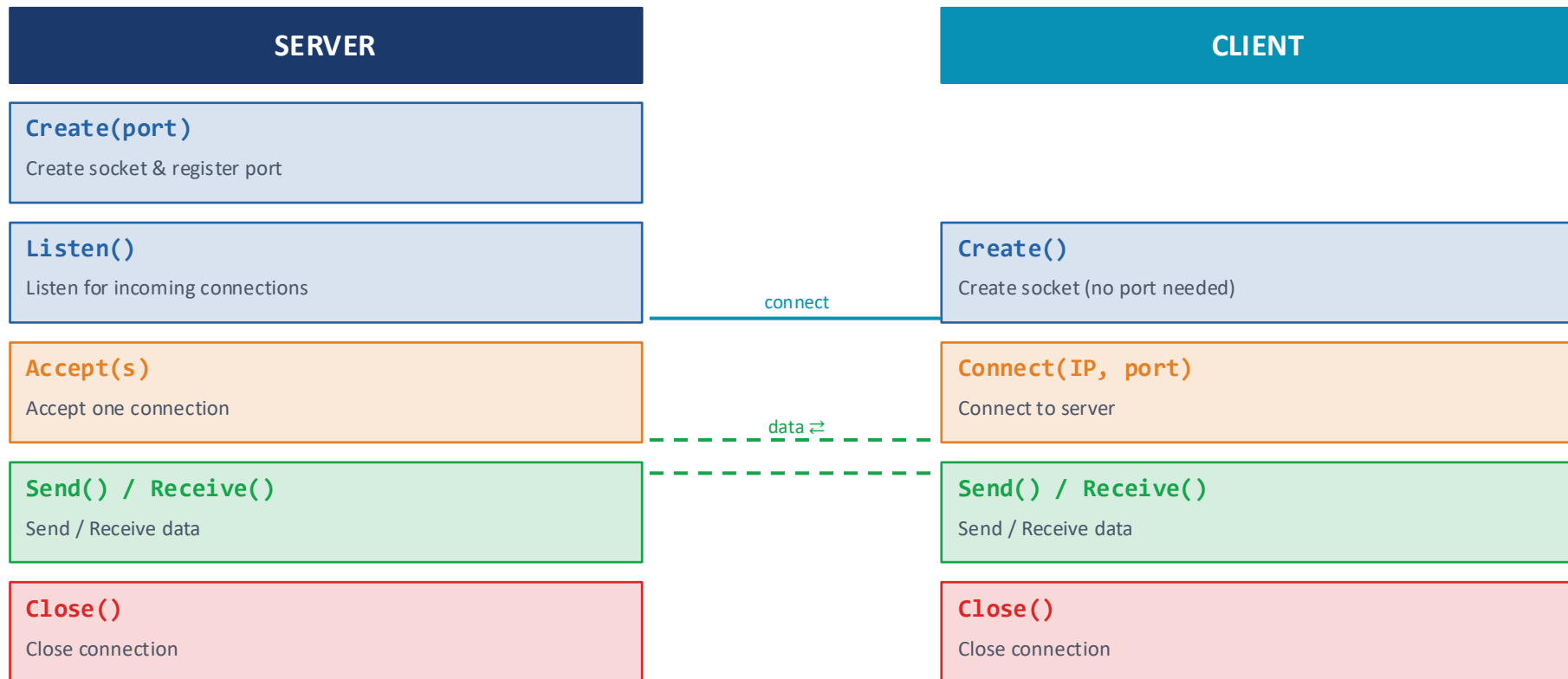


Languages & Platforms

C, C++, Java, Python, C#, VB, ... Cross-platform: Windows (Winsock), Linux/macOS (POSIX sockets)

Building a Client–Server Application

TCP Socket Workflow



Python – socket Library

import socket · Cross-platform: Windows / Linux / macOS

TCP Server

```
import socket
s = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM)
s.bind(('0.0.0.0', 1234))
s.listen(5)
conn, addr = s.accept()
data = conn.recv(1024)
conn.send(b'Hello!')
conn.close()
s.close()
```

TCP Client

```
import socket
c = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM)
c.connect(('127.0.0.1', 1234))
c.send(b'Hello Server!')
data = c.recv(1024)
print(data.decode())
c.close()
```

Key functions in the socket module

`socket(AF_INET, SOCK_STREAM|SOCK_DGRAM)`

create

Create a TCP (STREAM) or UDP (DGRAM) socket

`bind((host, port))`

server

Bind socket to a specific IP address and port

`listen(backlog)`

server

Start listening; backlog = max pending connection queue

`accept()`

server

Block and wait for a client → returns (conn_socket, addr)

`connect((host, port))`

client

Client connects to server at host:port

`send(data) / recv(size)`

TCP

Send/receive bytes over an established TCP connection

`sendto(data, addr) / recvfrom(size)`

UDP

Send/receive UDP datagrams (no connect required)

`close()`

close

Close socket and release resources

Demo – TCP Chat Application (1 Server – 1 Client)

Sequential chat: Server sends first

Scenario:

Server sends → Client receives & replies → Server receives & replies → ... (loop until either side types 'exit'). Message format: <length><message>

SERVER (afxsock.h)

```
CSocket server, s;  
AfxSocketInit(NULL);  
server.Create(1234);  
server.Listen();  
server.Accept(s);  
do {  
    printf("\nServer: ");  
    gets(s_str); len=strlen(s_str);  
    s.Send(s_str, len, 0);  
    len = s.Receive(r_str,100,0);  
    r_str[len] = 0;  
    printf("\nClient: %s", r_str);  
} while(strcmp(r_str,"exit") &&  
        strcmp(s_str,"exit"));  
s.Close(); server.Close();
```

CLIENT (afxsock.h)

```
CSocket client;  
AfxSocketInit(NULL);  
client.Create();  
client.Connect(svrAddr, 1234);  
do {  
    len=client.Receive(r_str,100,0);  
    r_str[len] = 0;  
    printf("\nServer: %s", r_str);  
    printf("\nClient: ");  
    gets(s_str);  
    client.Send(s_str,  
                strlen(s_str), 0);  
} while(strcmp(r_str,"exit") &&  
        strcmp(s_str,"exit"));  
client.Close();
```

Demo – UDP Chat Application (Python)

Connectionless · socket.SOCK_DGRAM · sendto / recvfrom

Scenario:

Server binds to a port and waits. Client sends a datagram, server receives it with `recvfrom()` (learns client address), replies with `sendto()`. Loop until either side sends 'exit'. No handshake required.

UDP SERVER (Python)

```
import socket

s = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM)
s.bind(('0.0.0.0', 1234))
print("UDP Server ready...")

while True:
    data, addr = s.recvfrom(1024)
    msg = data.decode()
    print(f"Client: {msg}")
    if msg == 'exit': break
    reply = input("Server: ")
    s.sendto(reply.encode(), addr)
    if reply == 'exit': break

s.close()
```

UDP CLIENT (Python)

```
import socket

c = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM)
srv = ('127.0.0.1', 1234)

while True:
    msg = input("Client: ")
    c.sendto(msg.encode(), srv)
    if msg == 'exit': break
    data, _ = c.recvfrom(1024)
    reply = data.decode()
    print(f"Server: {reply}")
    if reply == 'exit': break

c.close()
```



Key differences vs TCP: No `listen()/accept()` · Use `SOCK_DGRAM` · `sendto(data, addr)` & `recvfrom(size)` · No FIN on close

Demo – TCP chat vs UDP chat

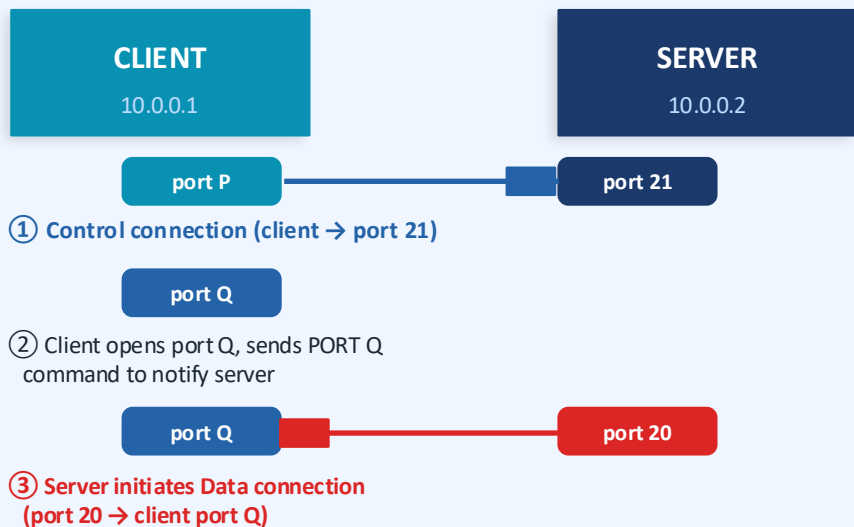
TCP Chat vs UDP Chat – Comparison:

Feature	TCP	UDP
Connection setup	socket() → bind() → listen() → accept()	socket() → bind() (server) socket() (client only)
Sending data	send(socket, data, len, 0)	sendto(socket, data, len, 0, addr, addrLen)
Receiving data	recv(socket, buf, len, 0)	recvfrom(socket, buf, len, 0, addr, &addrLen)
Termination	close(socket) [FIN exchange]	close(socket) [no FIN needed]
Reliability	✓ Guaranteed delivery	✗ Best-effort, packets may be lost
Best for	FTP, HTTP, Email, SSH	VoIP, DNS, Video streaming, Gaming

FTP – Active Mode vs Passive Mode

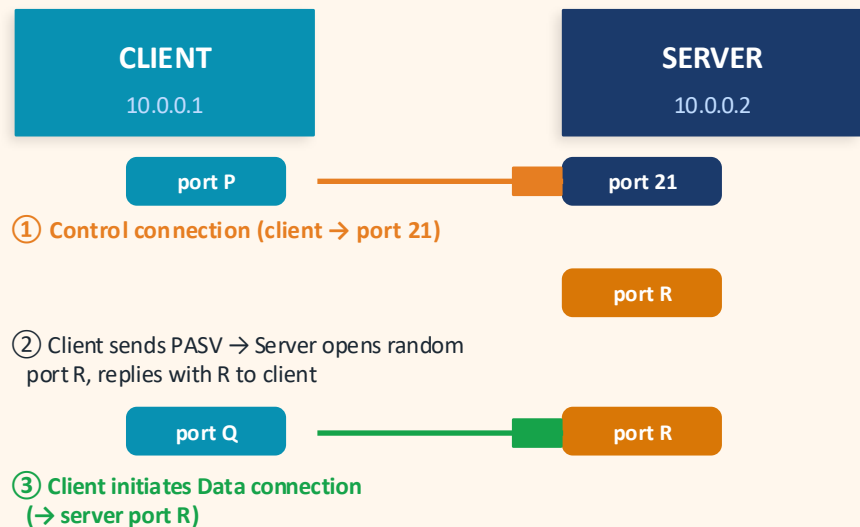
Connection diagram: Control & Data channels · RFC 959

ACTIVE MODE



⚠ Problem: Server initiates inbound connection to client → Firewall/NAT on client side typically blocks this → client must open port Q

PASSIVE MODE (PASV)



✓ Advantage: Client always initiates outbound connections → Firewall/NAT on client side does not block. Ideal for modern internet environments.

FTP – File Transfer Protocol Header

Application Layer · Port 20 (data) & Port 21 (control)

FTP Commands (Control Channel)

Port 21 · Control channel

USER <username>	Send login username
------------------------	---------------------

PASS <password>	Send password
------------------------	---------------

LIST	List files/directories
-------------	------------------------

RETR <filename>	Download file from server
------------------------	---------------------------

STOR <filename>	Upload file to server
------------------------	-----------------------

PASV	Switch to Passive mode
-------------	------------------------

QUIT	Terminate FTP session
-------------	-----------------------

FTP Response Codes

3-digit code + descriptive message

1xx	Positive Preliminary (in progress)
------------	------------------------------------

2xx	Positive Completion (success)
------------	-------------------------------

3xx	Positive Intermediate (more info needed)
------------	--

4xx	Transient Negative (temporary error)
------------	--------------------------------------

5xx	Permanent Negative (fatal error)
------------	----------------------------------

220	Service ready for new user
------------	----------------------------

230	User logged in successfully
------------	-----------------------------

226	Closing data connection, transfer OK
------------	--------------------------------------

550	File unavailable or access denied
------------	-----------------------------------

Summary

- ▶ OSI 7 layers & TCP/IP 4 layers – Socket operates at Transport layer
- ▶ TCP vs UDP – Reliability vs Speed; choose based on application needs
- ▶ TCP Header (20B) · UDP Header (8B) · FTP Command / Response codes
- ▶ Flow Control (rwnd) · Congestion Control (cwnd, ssthresh)
- ▶ Sequence Numbers · Timeout · Fast Retransmit · SACK
- ▶ Python socket API: socket() → bind() → listen() → accept() → send/recv → close()
- ▶ FTP: Active Mode (ports 20/21) vs Passive Mode (PASV) — firewall implications